Por que a IA repete o componente genérico

(e o que muda quando você especifica)

TDC Community · TDC Talks · quarta, 21/10/2026 · 19h às 20h30 (Brasília) · Thiago Xikota

“Moderno e responsivo” descreve qualidades, mas não distingue as alternativas que servem ao nosso produto. Neste exemplo, vamos observar quais escolhas a IA completou e o que muda quando a equipe fornece contexto, restrições e uma regra verificável.

Um formulário de convite, quatro apostas e uma decisão que ficou em aberto. Você sai com uma regra de produto, um custo assumido e um teste para levar ao próximo refinamento ou PR. Precisa só de navegador. Editor de código e ferramenta de IA são opcionais. Nenhum convite real é enviado.

Antes de começar

1. Abra **thiagoxikota.github.io/tdc-convite** no navegador do computador. A prática precisa de teclado. Sem internet, extraia o `kit-participante.zip` e abra `index.html`.
2. **Não abra a saída A ainda.** A primeira atividade é uma aposta.
3. Guarde sua resposta antes de ver o resultado. No bingo, mantenha as nove escolhas privadas e envie só o número de acertos depois da revelação. Nas outras apostas, responda ao sinal do facilitador.
4. Se algum arquivo não abrir, acompanhe pela tela do facilitador e anote no papel. A prática também funciona assim.

Aposta 1: bingo do componente genérico

O pedido A, na parte de produto, era só isto: *“Crie um componente moderno e responsivo para convidar uma pessoa para uma equipe.”* **Escolha exatamente nove casas.** Três das doze são armadilhas: não vieram. Você tem um minuto.

☐ **Sobretítulo em caixa alta**	☐ **Campo de mensagem pessoal**	☐ **Título motivacional**	☐ **Frase de efeito no card**
☐ **Card no centro, com sombra**	☐ **Foto ou avatar da pessoa**	☐ **Ícone num quadradinho**	☐ **Botão de largura total**
☐ **Setinha no botão**	☐ **Link de convite para copiar**	☐ **Tela de sucesso com check**	☐ **Membro / Administrador**

Marquei **9 de 12**. Depois da revelação: caí em _____ armadilha(s). Acertei **9 menos esse número**: _____ de 9. No chat, envie só esse total de acertos, ao sinal do facilitador.

Esse é um jogo de reconhecimento. O placar não mede com que frequência uma IA repete um padrão. Na revelação, veja o formulário e o estado de sucesso: uma imagem só não mostra todas as casas.

Aposta 2: quantos testes A passa?

Chute antes: 0, 1, 2 ou 3. Meu chute: _____

Abra `demonstracao/convite-a.html`. O **Painel do laboratório**, embaixo do formulário, mostra quantas chamadas a API recebeu, o último e-mail, o perfil e o estado. Recarregue a página antes de cada teste.

Teste	Faça	Passa?
C10 contexto	Olhe a página. Ela cabe dentro de uma gestão de pessoas que já existe?	
C3 espera	Envie `novo@example.com` e, durante 1,2 s, aperte Enter e clique de novo. Painel: quantas chamadas?	
C6 falha	Envie `falha@example.com`. Sem recarregar, envie de novo.	

Anote uma decisão que A tomou sem ninguém pedir: _____

No trabalho, esse contexto vem do repositório, do design system, dos contratos de API e de decisões anteriores da equipe. O refinamento decide o que ainda ficou aberto. Aqui, o HTML isolado é uma bancada para deixar os estados visíveis; você leva a decisão para a stack do seu produto.

Quem atua em produto pode explicitar intenção, regra, custo e resultado. Quem desenvolve pode confrontá-los com os estados, contratos e código. Os dois lados desafiam e testam as decisões; qualquer pessoa participa sem precisar implementar.

O pedido deixou decisões visuais abertas. Agora vamos procurar uma decisão de comportamento que também ficou aberta.

A saída B e a decisão D1

O pedido B (`demonstracao/pedido-b.txt`) traz contexto, perfis Leitor e Editor, dez critérios e uma decisão de produto, D1: *depois do sucesso, editar e-mail ou perfil limpa a confirmação anterior.* Abra `demonstracao/convite-b.html`.

Teste D1: envie `ana@example.com`, espere a confirmação e comece a apagar o e-mail. A frase sumiu?

____

Aposta 3: a janela

Escolha uma letra antes do teste. Envie `ana@example.com` com Leitor e, **durante a espera**, troque o perfil para Editor. Com o foco no e-mail, a sequência é **Enter, Tab e E**. A operação do select pode variar no seu navegador. Após duas tentativas sem capturar a janela, use o ensaio de quatro segundos e registre que observou a reprodução didática, separada de B original. Quem não conseguir abrir pode acompanhar a demonstração. O que aparece?

A. A confirmação nem aparece · B. Aparece “perfil Editor” · C. Aparece “perfil Leitor” com o select em Editor · D. Dá erro

Minha letra: ____ O que aconteceu: _____

Faça o mesmo em A. O que muda? _____

Separe as observações: o que foi enviado, o que o recibo confirma e o que os campos mostram agora. Uma diferença entre recibo e rascunho pede uma regra de apresentação; não significa que o payload enviado foi alterado.

Prática: sua regra sob teste (20 min)

Não abra os patches ainda. Primeiro decida o que a interface deve fazer quando alguém altera e-mail ou perfil durante o envio. Depois tente quebrar a decisão.

Trabalhe em dupla, se houver esse espaço. Na transmissão, escreva primeiro o seu cartão; a troca pelo chat tem um protocolo na etapa adversária. Sozinho, faça também o papel de revisor. Sem computador, acompanhe a demonstração e escreva a regra no papel.

1. Escreva seu cartão (5 min)

```
Contexto: [de onde vem a regra?]
Regra:    [o que aparece e o que fica editável?]
Custo:    [o que perdemos com essa escolha?]
Teste:    [passos] → [resultado esperado]
Em B:     [passa ou falha na regra escrita?]
Revisão:  [o que mudou após o contraexemplo?]
```

Escreva um resultado que outra pessoa consiga verificar. “Não fica confuso” ainda não diz o que testar. Nomeie o e-mail e o perfil do envio, os valores atuais dos campos e o que a confirmação deve dizer.

2. Tente quebrar a regra (5 min)

Troque os cartões. A dupla revisora propõe um caso e pergunta: o que acontece aqui? Pode ser editar duas vezes, mudar e voltar ao valor original, errar uma letra, receber falha ou tentar enviar outra vez. Não precisa encontrar defeito: uma regra que resiste também é resultado.

No remoto: ao sinal, publique **Regra + Custo**. O facilitador escolhe dois cartões e identifica seus autores. Escolha um deles e responda com **nome do cartão + passos que tentam quebrá-lo**. Quem não publicar pode revisar o próprio cartão ou um dos dois escolhidos.

Nosso contraexemplo: _____

A regra resistiu ou mudou? Por quê? _____

3. Teste e compare (8 min)

Rode os passos em B e registre o que aconteceu, **antes** de abrir a referência. B já confirma o payload enviado. Ele atende sua regra adicional de apresentação? Se sim, registre que passou e preserve o teste como regressão. Se sua regra exige outro comportamento, registre a diferença e o teste que falha. Não invente um defeito para justificar um patch.

Se após duas tentativas você não capturou a janela de B, use o ensaio de quatro segundos como apoio e anote **reprodução didática observada**. Esse ensaio mostra o comportamento da janela; ele não executa os patches nem substitui testes de falha ou outros contraexemplos.

Quando o facilitador liberar, abra as implementações de comparação. Rode os mesmos passos. Se sua regra for outra, escreva o esperado da sua versão; implementação é opcional. Não declare que ela passou sem executá-la.

Versão	Resultado observado	Atende à regra escrita?
B		
Implementação de comparação escolhida		
Outra regra, se houver	Escreva o esperado; implementação opcional	

4. Revise e defenda (2 min)

Complete a última linha do cartão. Prepare uma defesa de trinta segundos: o que sua regra garante e qual custo você aceita. O outro grupo pode pedir um contraexemplo. O cartão já contém o contexto, a decisão e a verificação para um refinamento ou PR.

Aposta 4: a busca

```
busca.addEventListener('input', async () => {
  const termo = busca.value;
  const pessoas = await buscarPessoas(termo);
  lista.render(pessoas, termo);
});
```

A equipe tem Ana, Anderson e Luana. A busca de “an” sai em **0 ms**; a de “ana”, **100 ms depois**. A primeira leva **900 ms** para responder; a segunda, **200 ms**. O que fica na tela no fim?

Qual regra você usaria aqui? □ trava □ recibo □ descarta a resposta velha □ outra: _____

Por quê? _____

Desafio sênior: o servidor gravou, a resposta se perdeu

Cenário para discussão, fora da API simulada. O servidor criou o convite da Ana. A resposta não chegou ao navegador, que só observou uma falha de rede.

O que a interface sabe com certeza? _____

O que ela mostra? Como consulta o resultado? Em que condição repetir o envio é seguro?

_____

Diga qual parte dessa decisão depende do backend: _____

No próximo refinamento ou PR

1. Que regra falta e de onde vem o contexto?
2. Podemos descartar a resposta ou a ação já tem efeito?
3. Que teste verifica essa regra e seu custo?

Leve para o próximo refinamento ou PR

Escolha um componente real do seu produto. Escreva uma regra que ainda falta decidir, de onde vem o contexto, o custo e um teste observável. Pode usar este cartão:

```
Componente: [qual componente e onde é usado]
Regra ausente: [a decisão que falta]
Fonte do contexto: [arquivo, design system, contrato ou decisão da equipe]
Custo: [o que a regra dificulta ou exige]
Teste: [passos] → [resultado esperado]
```

Guarde o cartão no refinamento, ticket ou PR. Produto e engenharia podem começar a conversa por ele; implementar é um próximo passo, não uma condição para participar.

Com IA, se quiser implementar depois: cole seu cartão no fim de `demonstracao/pedido-b.txt`, use conversa nova e espere no máximo 4 minutos. Salve a primeira saída como `meu-convite.html`, registre a ferramenta e rode o mesmo teste. Se a IA alterar a API simulada ou o painel, o resultado não é comparável.

Respostas das apostas, código e patches explicados, depois da atividade. Ferramenta de ensaio e visualização, para acompanhar a demonstração quando o facilitador liberar. A instrumentação do ensaio é separada das saídas originais.

Os links do PDF abrem o kit online. Para usar sem internet, extraia o ZIP e abra `index.html`.

Procedência e limites

De onde vêm A e B. Duas sessões separadas do Codex em 19/10/2026, uma geração por condição, mesma base, primeira saída guardada sem edição. O modelo não foi informado pela ferramenta. Pedidos integrais e registro em `demonstracao/`. Um par de saídas e o bingo não medem frequência nem isolam uma causa geral para a repetição.

O que as comparações são. Os dois patches em `kit-tdc/` são cópias de B com alterações escritas à mão para a aula, rotuladas na própria página. A referência completa também é didática. Nenhum deles é saída gerada.

O que o laboratório não prova. A API simulada (`window.apiConvite`, 1,2 s de espera) não tem rede, servidor nem autorização: aceita o perfil que recebe. Bloquear o botão evita chamadas concorrentes nessa página; não protege por si só contra recarga ou resposta perdida. Uma nova tentativa segura pode exigir uma chave de idempotência, com deduplicação no servidor e consulta do resultado. `falha@example.com` simula uma falha conhecida, não uma escrita de resultado desconhecido. Leitor de tela e zoom nativo não foram testados.

Os dez critérios do pedido B, para consulta

ID	Critério
C1	E-mail vazio ou inválido não chama a API; erro junto ao campo e foco nele.
C2	Leitor por padrão; só Leitor e Editor.
C3	Andamento visível na espera e nenhuma chamada duplicada.
C4	Confirmação mostra e-mail e perfil enviados.
C5	`existe@example.com` explica que a pessoa já está na equipe e preserva os campos.
C6	Falha preserva campos; nova tentativa funciona.
C7	Teclado opera tudo, com foco visível; resposta não rouba o foco de quem saiu.
C8	Labels, erro associado, `aria-invalid`, região de status.
C9	Sem corte em 320 px e zoom de 200%.
C10	Bloco integrado à gestão de pessoas, tokens existentes, sem hero, sidebar ou avatar.

Fontes: WCAG 2.2, teclado · W3C WAI, notificações de formulário · RFC 9110, métodos idempotentes · MDN, AbortController